

Assignment 4: Routing and State Management

Subject: ReactJS | Unit 4: Routing and State Management

B.Sc. Computer Science

Q1. Single Page Application (SPA) using React Router

A Single Page Application loads one HTML page and dynamically updates content without full page reloads. React Router enables client-side routing. Install it with: `npm install react-router-dom`

Program / Code:

```
// Step 1: Install React Router
// npm install react-router-dom

// App.js
import React from 'react';
import { BrowserRouter, Routes, Route, Link } from 'react-router-dom';

// Page components
function Home() { return <h2>Home Page - Welcome to SPA!</h2>; }
function About() { return <h2>About Page - Learn about us.</h2>; }
function Contact() { return <h2>Contact Page - Reach us here.</h2>; }

function App() {
  return (
    <BrowserRouter>
      <nav>
        <Link to='/'>Home</Link> |
        <Link to='/about'>About</Link> |
        <Link to='/contact'>Contact</Link>
      </nav>
      <hr />
      <Routes>
        <Route path='/' element={<Home />} />
        <Route path='/about' element={<About />} />
        <Route path='/contact' element={<Contact />} />
      </Routes>
    </BrowserRouter>
  );
}

export default App;
```

Expected Output:

```
Output (URL: /)          -> Home Page - Welcome to SPA!
Output (URL: /about)     -> About Page - Learn about us.
Output (URL: /contact)   -> Contact Page - Reach us here.
Navigation: Home | About | Contact (no page reload on click)
```

Q2. Configure React Router in a Project

React Router v6 uses `BrowserRouter` to wrap the app, `Routes` to define the routing context, and `Route` to map URL paths to components. The `Link` component replaces anchor tags for SPA navigation.

Program / Code:

```

// index.js - Wrap App with BrowserRouter at root level
import React from 'react';
import ReactDOM from 'react-dom/client';
import { BrowserRouter } from 'react-router-dom';
import App from './App';

ReactDOM.createRoot(document.getElementById('root')).render(
  <BrowserRouter>
    <App />
  </BrowserRouter>
);

// App.js - Define routes
import React from 'react';
import { Routes, Route, Link, NavLink } from 'react-router-dom';

function Home() { return <h2>Home Page</h2>; }
function Services(){ return <h2>Services Page</h2>; }
function Portfolio(){ return <h2>Portfolio Page</h2>; }

function App() {
  return (
    <div>
      <nav>
        { /* NavLink adds 'active' class to current route */ }
        <NavLink to="/">Home</NavLink>
        <NavLink to="/services">Services</NavLink>
        <NavLink to="/portfolio">Portfolio</NavLink>
      </nav>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/services" element={<Services />} />
        <Route path="/portfolio" element={<Portfolio />} />
      </Routes>
    </div>
  );
}

export default App;

```

Expected Output:

Output: Navigation bar with 3 links
 Home | Services | Portfolio
 Clicking each link updates the URL and renders the matching component
 NavLink auto-applies 'active' CSS class to the current page link

Q3. Navigate Between Home, About and Contact Pages

We create separate component files for each page and use React Router's Link component for navigation. useNavigate() hook allows programmatic navigation (redirecting inside functions, not just from clicks).

Program / Code:

```
import React from 'react';
import { BrowserRouter, Routes, Route,
      Link, useNavigate } from 'react-router-dom';

function Navbar() {
  return (
    <nav style={{ background:'#1a237e', padding:'10px' }}>
      <Link to='/' style={{color:'white', margin:'10px'}}>Home</Link>
      <Link to='/about' style={{color:'white', margin:'10px'}}>About</Link>
      <Link to='/contact' style={{color:'white', margin:'10px'}}>Contact</Link>
    </nav>
  );
}

function Home() {
  const navigate = useNavigate();
  return (
    <div>
      <h2>Home Page</h2>
      <p>Welcome to our website!</p>
      <button onClick={() => navigate('/about')}>Go to About</button>
    </div>
  );
}

function About() {
  return (<div><h2>About Page</h2><p>We are a React learning team.</p></div>);
}

function Contact() {
  return (<div><h2>Contact Page</h2><p>Email: info@react.com</p></div>);
}

function App() {
  return (
    <BrowserRouter>
      <Navbar />
      <Routes>
        <Route path='/' element={<Home />} />
        <Route path='/about' element={<About />} />
        <Route path='/contact' element={<Contact />} />
      </Routes>
    </BrowserRouter>
  );
}

export default App;
```

Expected Output:

```
Output: Blue navbar: Home | About | Contact
/       -> Home Page + 'Go to About' button
/about  -> About Page - We are a React learning team.
/contact -> Contact Page - Email: info@react.com
Clicking 'Go to About' button navigates programmatically via useNavigate()
```

Q4. Dynamic Routing in React Router

Dynamic routes use URL parameters (prefixed with :) to capture values from the URL. The useParams() hook reads these values inside the component. Example: /student/21 captures id=21 as a route parameter.

Program / Code:

```
import React from 'react';
import { BrowserRouter, Routes, Route,
      Link, useParams } from 'react-router-dom';

const students = [
  { id: 1, name: 'Rahul Patil',   course: 'B.Sc. CS' },
  { id: 2, name: 'Priya Sharma', course: 'BCA'      },
  { id: 3, name: 'Amit Desai',   course: 'MCA'      },
];

function StudentList() {
  return (
    <div>
      <h2>Student List</h2>
      {students.map(s => (
        <div key={s.id}>
          <Link to={`/student/${s.id}`}>{s.name}</Link>
        </div>
      ))}
    </div>
  );
}

function StudentDetail() {
  const { id } = useParams(); // Read :id from URL
  const student = students.find(s => s.id === Number(id));
  if (!student) return <h3>Student not found!</h3>;
  return (
    <div>
      <h2>Student Detail</h2>
      <p>ID      : {student.id}</p>
      <p>Name    : {student.name}</p>
      <p>Course  : {student.course}</p>
      <Link to="/">Back to List</Link>
    </div>
  );
}

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<StudentList />} />
        <Route path="/student/:id" element={<StudentDetail />} />
      </Routes>
    </BrowserRouter>
  );
}

export default App;
```

Expected Output:

```
Output (URL: /)      -> Student List: Rahul | Priya | Amit
Output (URL: /student/1) -> Student Detail:
                        ID: 1 | Name: Rahul Patil | Course: B.Sc. CS
Output (URL: /student/2) -> Name: Priya Sharma | Course: BCA
```

Q5. Nested Routes in React Router

Nested routes allow child routes to render inside parent components. The parent route uses Outlet from react-router-dom as a placeholder where child route components are rendered.

Program / Code:

```
import React from 'react';
import { BrowserRouter, Routes, Route,
      Link, Outlet } from 'react-router-dom';

// Parent layout component - contains Outlet
function Dashboard() {
  return (
    <div>
      <h2>Dashboard</h2>
      <nav>
        <Link to='profile'>Profile</Link> |
        <Link to='settings'>Settings</Link> |
        <Link to='reports'>Reports</Link>
      </nav>
      <hr />
      { /* Child routes render here */ }
      <Outlet />
    </div>
  );
}

// Child components
function Profile() { return <h3>Profile: Rahul Patil, TY B.Sc. CS</h3>; }
function Settings() { return <h3>Settings: Theme - Dark, Lang - English</h3>; }
function Reports() { return <h3>Reports: Attendance 92%, Marks 85%</h3>; }
function DashHome() { return <h3>Welcome to Dashboard! Select a menu.</h3>; }

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path='/dashboard' element={<Dashboard />}>
          { /* Nested (child) routes */ }
          <Route index element={<DashHome />} />
          <Route path='profile' element={<Profile />} />
          <Route path='settings' element={<Settings />} />
          <Route path='reports' element={<Reports />} />
        </Route>
      </Routes>
    </BrowserRouter>
  );
}
export default App;
```

Expected Output:

Output (URL: /dashboard)	-> Dashboard + Welcome to Dashboard!
Output (URL: /dashboard/profile)	-> Dashboard + Profile: Rahul Patil
Output (URL: /dashboard/settings)	-> Dashboard + Settings: Theme - Dark
Output (URL: /dashboard/reports)	-> Dashboard + Attendance 92%

Q6. Handle Invalid URL – 404 Not Found Page

React Router v6 uses a catch-all route with path='*' to handle unmatched URLs. This wildcard route renders a custom 404 component when no other route matches the current URL.

Program / Code:

```
import React from 'react';
import { BrowserRouter, Routes, Route,
      Link, useNavigate } from 'react-router-dom';

function Home() { return <h2>Home Page</h2>; }
function About() { return <h2>About Page</h2>; }

// 404 Component
function NotFound() {
  const navigate = useNavigate();
  return (
    <div style={{ textAlign:'center', marginTop:'50px' }}>
      <h1 style={{ fontSize:'72px', color:'#e53935' }}>404</h1>
      <h2>Page Not Found!</h2>
      <p>The page you are looking for does not exist.</p>
      <button onClick={() => navigate('/')}>
        Go Back to Home
      </button>
    </div>
  );
}

function App() {
  return (
    <BrowserRouter>
      <nav>
        <Link to='/'>Home</Link> |
        <Link to='/about'>About</Link> |
        <Link to='/xyz'>Invalid Link</Link>
      </nav>
      <Routes>
        <Route path='/' element={<Home />} />
        <Route path='/about' element={<About />} />
        /* Catch-all: matches any unrecognized URL */
        <Route path='*' element={<NotFound />} />
      </Routes>
    </BrowserRouter>
  );
}

export default App;
```

Expected Output:

```
Output (URL: /)      -> Home Page
Output (URL: /about) -> About Page
Output (URL: /xyz)   -> 404 (large red text)
                      Page Not Found!
                      The page you are looking for does not exist.
                      [ Go Back to Home ]
```

Q7. Pass Route Parameters using React Router

Route parameters are defined in the path using `:paramName` syntax. `useParams()` hook reads them inside the component. `useSearchParams()` reads query string parameters like `?id=1&name;=Rahul`.

Program / Code:

```
import React from 'react';
import { BrowserRouter, Routes, Route,
      Link, useParams, useSearchParams } from 'react-router-dom';

// Route param: /product/:category/:id
function ProductDetail() {
  const { category, id } = useParams();
  return (
    <div>
      <h2>Product Detail</h2>
      <p>Category : {category}</p>
      <p>ID       : {id}</p>
    </div>
  );
}

// Query params: /search?term=react&page=1
function SearchPage() {
  const [searchParams] = useSearchParams();
  const term = searchParams.get('term');
  const page = searchParams.get('page');
  return (
    <div>
      <h2>Search Results</h2>
      <p>Search Term : {term}</p>
      <p>Page       : {page}</p>
    </div>
  );
}

function App() {
  return (
    <BrowserRouter>
      <nav>
        <Link to='/product/electronics/101'>Product 101</Link> |
        <Link to='/search?term=ReactJS&page=1'>Search</Link>
      </nav>
      <Routes>
        <Route path='/product/:category/:id' element={<ProductDetail />} />
        <Route path='/search'                element={<SearchPage />} />
      </Routes>
    </BrowserRouter>
  );
}
export default App;
```

Expected Output:

```
Output (URL: /product/electronics/101):
  Category: electronics | ID: 101
Output (URL: /search?term=ReactJS&page=1):
  Search Term: ReactJS | Page: 1
```

Q8. Conditional Routing in React

Conditional routing shows different routes based on app state such as authentication. A Protected Route wrapper checks if the user is logged in and either renders the page or redirects to the login screen.

Program / Code:

```

import React, { useState } from 'react';
import { BrowserRouter, Routes, Route,
      Link, Navigate } from 'react-router-dom';

// Protected Route wrapper
function ProtectedRoute({ isLoggedIn, children }) {
  if (!isLoggedIn) {
    return <Navigate to='/login' replace />;
  }
  return children;
}

function LoginPage({ onLogin }) {
  return (
    <div>
      <h2>Login Page</h2>
      <button onClick={onLogin}>Login as Admin</button>
    </div>
  );
}

function Dashboard() {
  return <h2>Dashboard - Protected Page (Logged In Users Only)</h2>;
}

function Profile() {
  return <h2>Profile - Protected Page</h2>;
}

function App() {
  const [isLoggedIn, setIsLoggedIn] = useState(false);
  return (
    <BrowserRouter>
      <nav>
        <Link to='/'>Home</Link> |
        <Link to='/dashboard'>Dashboard</Link> |
        <Link to='/profile'>Profile</Link>
        {isLoggedIn &&
          <button onClick={() => setIsLoggedIn(false)}>Logout</button>
        }
      </nav>
      <Routes>
        <Route path='/' element={<h2>Home Page</h2>} />
        <Route path='/login'
          element={<LoginPage onLogin={() => setIsLoggedIn(true)} />} />
        <Route path='/dashboard' element={
          <ProtectedRoute isLoggedIn={isLoggedIn}>
            <Dashboard />
          </ProtectedRoute>
        } />
        <Route path='/profile' element={
          <ProtectedRoute isLoggedIn={isLoggedIn}>
            <Profile />
          </ProtectedRoute>
        } />
      </Routes>
    </BrowserRouter>
  );
}
export default App;

```

Expected Output:

```
Output (not logged in, click Dashboard) -> Redirects to /login
Output (click 'Login as Admin')         -> isLoggedIn = true
Output (click Dashboard after login)     -> Dashboard page renders
Output (click Logout, click Profile)     -> Redirects to /login again
```

Q9. State Lifting Between Components

State lifting means moving state up to a common parent so multiple child components can share and update the same data. The parent holds the state and passes it down via props along with handler functions.

Program / Code:

```
import React, { useState } from 'react';

// Child A: displays temperature in Celsius
function CelsiusInput({ celsius, onChange }) {
  return (
    <div>
      <label>Celsius: </label>
      <input
        type='number'
        value={celsius}
        onChange={(e) => onChange(e.target.value)}
      />
    </div>
  );
}

// Child B: displays temperature in Fahrenheit (read-only)
function FahrenheitDisplay({ fahrenheit }) {
  return (
    <div>
      <label>Fahrenheit: </label>
      <input type='number' value={fahrenheit} readOnly
        style={{ background: '#f0f0f0' }} />
    </div>
  );
}

// Parent: holds the LIFTED state
function TemperatureConverter() {
  const [celsius, setCelsius] = useState(0);
  const fahrenheit = (celsius * 9/5 + 32).toFixed(2);

  return (
    <div>
      <h2>Temperature Converter (State Lifting)</h2>
      <CelsiusInput celsius={celsius} onChange={setCelsius} />
      <FahrenheitDisplay fahrenheit={fahrenheit} />
      <p>Formula: F = (C x 9/5) + 32</p>
    </div>
  );
}

export default TemperatureConverter;
```

Expected Output:

```
Output (celsius = 0)    : Fahrenheit = 32.00
Output (celsius = 100)  : Fahrenheit = 212.00
Output (celsius = 37)   : Fahrenheit = 98.60 (body temperature)
Both inputs stay in sync – state is lifted to parent component.
```

Q10. Global State using Context API

Context API allows sharing data globally without prop drilling. `createContext()` creates a context, `Provider` wraps the component tree and supplies the value, and `useContext()` reads it in any child component.

Program / Code:

```
import React, { createContext, useContext, useState } from 'react';

// Step 1: Create Context
const ThemeContext = createContext();

// Step 2: Create a custom hook for convenience
const useTheme = () => useContext(ThemeContext);

// Deep child component - reads theme directly, no prop drilling!
function Header() {
  const { theme } = useTheme();
  return (
    <header style={{
      background: theme === 'dark' ? '#1e1e2e' : '#e3f2fd',
      color:      theme === 'dark' ? 'white'   : 'black',
      padding: '10px'
    }}>
      <h2>Header - Theme: {theme}</h2>
    </header>
  );
}

function Content() {
  const { theme, toggleTheme } = useTheme();
  return (
    <main style={{
      background: theme === 'dark' ? '#2d2d2d' : '#ffffff',
      color:      theme === 'dark' ? '#eee'    : '#333',
      padding: '20px'
    }}>
      <p>Content area. Current theme: <b>{theme}</b></p>
      <button onClick={toggleTheme}>Toggle Theme</button>
    </main>
  );
}

// Step 3: Wrap with Provider
function App() {
  const [theme, setTheme] = useState('light');
  const toggleTheme = () =>
    setTheme(prev => prev === 'light' ? 'dark' : 'light');

  return (
    <ThemeContext.Provider value={{ theme, toggleTheme }}>
      <Header />
      <Content />
    </ThemeContext.Provider>
  );
}

export default App;
```

Expected Output:

Output (light theme): Header and Content have light background
Output (dark theme): Header and Content switch to dark colors
No props passed between components - Context provides global state.

Q11. Context Provider and Consumer

The Provider component makes context value available to all descendants. There are two ways to consume context: the older Consumer component pattern and the modern useContext hook. Below demonstrates both approaches.

Program / Code:

```
import React, { createContext, useContext } from 'react';

// Create User Context
const UserContext = createContext();

// Consumer using useContext hook (Modern - Recommended)
function UserCard() {
  const user = useContext(UserContext);
  return (
    <div style={{ border: '1px solid #ccc', padding: '10px', margin: '10px' }}>
      <h3>User Card (useContext hook)</h3>
      <p>Name : {user.name}</p>
      <p>Email : {user.email}</p>
      <p>Role : {user.role}</p>
    </div>
  );
}

// Consumer using Consumer component (Older pattern)
function UserBadge() {
  return (
    <UserContext.Consumer>
      {user => (
        <div style={{ background: '#e3f2fd', padding: '10px', margin: '10px' }}>
          <h3>User Badge (Consumer component)</h3>
          <p>{user.name} | {user.role}</p>
        </div>
      )}
    </UserContext.Consumer>
  );
}

// Provider wraps everything
function App() {
  const userValue = {
    name: 'Rahul Patil',
    email: 'rahul@example.com',
    role: 'Student'
  };

  return (
    <UserContext.Provider value={userValue}>
      <h2>Context Provider and Consumer</h2>
      <UserCard />
      <UserBadge />
    </UserContext.Provider>
  );
}

export default App;
```

Expected Output:

Output:

```
[ User Card (useContext hook) ]  
Name: Rahul Patil | Email: rahul@example.com | Role: Student  
[ User Badge (Consumer component) ]  
Rahul Patil | Student
```

Q12. useState Hook – Counter Application

useState is the most basic React Hook. It returns a stateful value and a setter function. When the setter is called, React re-renders the component with the new value. Multiple useState calls can manage different state values.

Program / Code:

```
import React, { useState } from 'react';  
  
function CounterApp() {  
  const [count, setCount] = useState(0);  
  const [step, setStep] = useState(1);  
  const [history, setHistory] = useState([]);  
  const [maxCount, setMaxCount] = useState(0);  
  
  const increment = () => {  
    const newVal = count + step;  
    setCount(newVal);  
    setHistory(prev => [...prev, `+${step} -> ${newVal}`]);  
    if (newVal > maxCount) setMaxCount(newVal);  
  };  
  
  const decrement = () => {  
    const newVal = count - step;  
    setCount(newVal);  
    setHistory(prev => [...prev, `-${step} -> ${newVal}`]);  
  };  
  
  const reset = () => {  
    setCount(0);  
    setHistory([]);  
  };  
  
  return (  
    <div>  
      <h2>useState Hook Counter</h2>  
      <h1 style={{ color: count < 0 ? 'red' : count > 0 ? 'green' : 'gray' }}>  
        {count}  
      </h1>  
      <label>Step: </label>  
      <input type='number' value={step} min='1'  
        onChange={e => setStep(Number(e.target.value))} />  
      <br />  
      <button onClick={increment}>+ {step}</button>  
      <button onClick={decrement}>- {step}</button>  
      <button onClick={reset}>Reset</button>  
      <p>Max reached: {maxCount}</p>  
      <p>History (last 5): {history.slice(-5).join(' | ')}</p>  
    </div>  
  );  
}
```

```
export default CounterApp;
```

Expected Output:

```
Output (initial):  0 (gray)  Step: 1
Output (+1, +1):   2 (green) | Max: 2 | History: +1->1 | +1->2
Output (-1, -1, -1): -1 (red) | History: ...+1->2 | -1->1 | -1->0 | -1->-1
```

Q13. Fetch Data using useEffect Hook

useEffect runs side effects after render. It is ideal for API calls, subscriptions, and DOM manipulation. The dependency array controls when it runs: [] means only on mount, [dep] means whenever dep changes.

Program / Code:

```
import React, { useState, useEffect } from 'react';

function DataFetcher() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    // Runs once after component mounts (empty dependency array)
    fetch('https://jsonplaceholder.typicode.com/users?_limit=5')
      .then(res => {
        if (!res.ok) throw new Error('Network error');
        return res.json();
      })
      .then(data => {
        setUsers(data);
        setLoading(false);
      })
      .catch(err => {
        setError(err.message);
        setLoading(false);
      });
  }, []); // Empty array = run once on mount

  if (loading) return <p>Loading data...</p>;
  if (error) return <p>Error: {error}</p>;

  return (
    <div>
      <h2>Users (fetched with useEffect)</h2>
      <ul>
        {users.map(user => (
          <li key={user.id}>
            {user.name} - {user.email}
          </li>
        ))}
      </ul>
    </div>
  );
}

export default DataFetcher;
```

Expected Output:

```
Output (loading): Loading data...
Output (success):
  Users (fetched with useEffect)
  - Leanne Graham - Sincere@april.biz
  - Ervin Howell - Shanna@melissa.tv
  - Clementine Bauch - Nathan@yesenia.net
  (5 users fetched from JSONPlaceholder API)
```

Q14. Create a Custom Hook

Custom Hooks are JavaScript functions that start with 'use' and can call other hooks. They let you extract reusable stateful logic from components. Below we create useLocalInput, useToggle, and useWindowSize custom hooks.

Program / Code:

```
import React, { useState, useEffect } from 'react';

// Custom Hook 1: useToggle
function useToggle(initialValue = false) {
  const [value, setValue] = useState(initialValue);
  const toggle = () => setValue(prev => !prev);
  return [value, toggle];
}

// Custom Hook 2: useInput (manages a single input field)
function useInput(initialValue = '') {
  const [value, setValue] = useState(initialValue);
  const onChange = (e) => setValue(e.target.value);
  const reset = () => setValue(initialValue);
  return { value, onChange, reset };
}

// Custom Hook 3: useWindowSize
function useWindowSize() {
  const [size, setSize] = useState({
    width: window.innerWidth,
    height: window.innerHeight
  });
  useEffect(() => {
    const handler = () =>
      setSize({ width: window.innerWidth, height: window.innerHeight });
    window.addEventListener('resize', handler);
    return () => window.removeEventListener('resize', handler);
  }, []);
  return size;
}

// Using all custom hooks in one component
function App() {
  const [isVisible, toggleVisible] = useToggle(true);
  const nameInput = useInput('Rahul');
  const { width, height } = useWindowSize();

  return (
    <div>
      <h2>Custom Hooks Demo</h2>
      <button onClick={toggleVisible}>
        {isVisible ? 'Hide' : 'Show'} Content
      </button>
      {isVisible && <p>Content is visible!</p>}
      <br />
      <input {...nameInput} placeholder='Name' />
      <button onClick={nameInput.reset}>Reset</button>
      <p>Typed: {nameInput.value}</p>
      <p>Window: {width} x {height}</p>
    </div>
  );
}

export default App;
```

Expected Output:

Output:

[Hide Content] -> Content is visible!

[Show Content] -> (content hidden)

Name input: [Rahul] [Reset]

Typed: Rahul

Window: 1366 x 768 (updates on resize)

Q15. Focus Input using useRef Hook

useRef() creates a persistent reference to a DOM element. Unlike useState, updating a ref does not trigger a re-render. Here we use refs to focus inputs, measure element dimensions, and store a previous value without causing re-renders.

Program / Code:

```
import React, { useRef, useState, useEffect } from 'react';

function UseRefDemo() {
  const firstNameRef = useRef(null);
  const lastNameRef = useRef(null);
  const renderCount = useRef(0); // Store value without re-render
  const [name, setName] = useState('');

  // Auto-focus first input when component mounts
  useEffect(() => {
    firstNameRef.current.focus();
  }, []);

  // Track renders using ref (NOT state - avoids infinite loop)
  useEffect(() => {
    renderCount.current += 1;
  });

  const handleSubmit = () => {
    const fullName = firstNameRef.current.value +
      ' ' + lastNameRef.current.value;
    setName(fullName);
    firstNameRef.current.value = '';
    lastNameRef.current.value = '';
    firstNameRef.current.focus(); // Re-focus after submit
  };

  return (
    <div>
      <h2>useRef Hook - Focus Input</h2>
      <input ref={firstNameRef} placeholder='First Name' />
      <input ref={lastNameRef} placeholder='Last Name'
        onPress={e => e.key==='Enter' && handleSubmit()} />
      <button onClick={handleSubmit}>Submit</button>
      {name && <p>Hello, <b>{name}</b>!!</p>}
      <p>Render count: {renderCount.current}</p>
    </div>
  );
}

export default UseRefDemo;
```

Expected Output:

```
Output (on mount):      Cursor auto-focuses in First Name field
Output (fill Rahul Patil, click Submit):
                        Hello, Rahul Patil!
                        Fields cleared, focus returns to First Name
Render count increments on each state change but NOT on ref updates.
```

End of Assignment 4 — Routing and State Management Topics: React Router v6 (BrowserRouter, Routes, Route, Link, NavLink, useNavigate, useParams, Outlet, Navigate), Context API, useState, useEffect, useRef, Custom Hooks, State Lifting, Protected Routes.